

UPPALA SRI CHAITANYA

Hyderabad, India

✉ uppalasrichaitanya2007@gmail.com  [linkedin](#)  [github](#)

Education

Keshav Memorial College of Engineering

B.Tech in Computer Science & Machine Learning (CSM)

Aug 2024 – May 2028 (Expected)

Hyderabad, India

Experience

AI Research Intern

Sep 2025 – Present

Teleparadigm Networks

- Designed and built **ServoPilot**, a desktop application for real-time configuration, PID tuning, and control of ST3215 smart servo motors, including a ROS2 bridge, live telemetry, and a packet-level protocol console.
- Adopted as the lab's primary tool for servo control and multi-servo synchronization.
- Co-authored a paper on the system, submitted to IEEE INDICON and currently under peer review.

Projects

Quiver | *Rust*

Vector database from scratch · [GitHub](#)

- Built a single-node vector database from scratch in Rust: mmap-backed storage, write-ahead log with crash recovery, multi-layer HNSW indexing, SQ8 scalar quantization, and hand-written x86_64 SIMD (AVX2/FMA) distance kernels.
- Made filtered search selectivity-aware during graph traversal instead of post-filtering; on SIFT1M held ≥ 0.9837 recall across 1–50% filter selectivity and narrowed the high/low-selectivity throughput gap from $\sim 25\times$ to $\sim 11\times$.

DUM-E | *Electron*

dum-e-lab.com | [GitHub](#)

- Built a desktop orchestrator running multiple coding-agent CLIs (Qwen, Claude, Codex, OpenCode) as concurrent live PTY sessions (xterm.js), each with an isolated working directory, persistent memory, and message inbox/outbox; a coordinating agent assigns and tracks work on a shared task board.
- Added cost guardrails (token/spend caps, circuit breaker) and scheduled/webhook/Slack triggers; released v0.1.1 for Windows and Linux.

Design-Taste MCP | *TypeScript*

npm | [GitHub](#)

- Built and published (v1.0.1, MIT) an MCP server with 7 tools giving AI coding agents design guidance: planning rubrics, static analysis and accessibility testing (axe-core), a component registry, and automated visual critique via vision models.
- Integrated 6 vision-model provider adapters with a documented fallback order; wrote a ~ 50 -assertion acceptance suite that spawns a real MCP server/client over stdio and exercises live vision-API calls.

Cartograph | *Next.js* / *TypeScript*

Live demo | [GitHub](#)

- Built a static-analysis tool that parses JavaScript, TypeScript, Python, and Go repositories into interactive dependency graphs, with every edge traced to an actual import statement rather than inferred from naming.
- Tagged every graph fact with a confidence level (verified/derived/heuristic/unknown), gated AI-generated explanations on citations to measured evidence, and sandboxed uploads (25MB / 800-file limit, no code execution).

Technical Skills

Programming Languages: TypeScript, JavaScript, Python, Rust, Java

Frameworks & Libraries: React, Next.js, Node.js, Electron, Axum (Rust), PyO3

Systems & Backend: mmap storage, write-ahead logging, SIMD optimization, HNSW/ANN search

AI/ML & Agent Systems: Model Context Protocol (MCP), multi-agent orchestration, LLM tool-use integration, vision-model APIs (OpenAI, Anthropic, Gemini)

Robotics & Embedded: ROS2, PID control, real-time telemetry